Entropy Weight Allocation: Positive-unlabeled Learning via Optimal Transport

Wen Gu* Teng Zhang* Hai Jin*

Abstract

Positive-unlabeled learning (PU learning) aims to deal with the problem that only a fraction of positive instances are known. Due to the absence of negative instances, ordinary learning models cannot be directly applied. Existing PU learning methods either explicitly choose some unlabeled instances as negative instances in advance or reformulate the task as a weighted learning problem. Since working in such an ad-hoc fashion, these methods often suffer a bad performance and only have limited usage. This paper proposes a novel instance-dependent weighting method *entropy weight allocation* (EWA) for PU learning by *optimal transport* (OT). More specifically, we allocate each unlabeled instance an elaborate weight indicating the possibility that it is an underlying negative instance. Then any ordinary weighted learning models can be used to obtain a PU classifier. By concatenating EWA with four celebrated classification models, we show that EWA is a broad-spectrum weighting method that can boost almost all the mainstream machine learning models for PU learning.

1 Introduction

With only positive instances and many unlabeled instances, the ordinary binary classification model cannot be directly applied due to the absence of negative instances. This scenario is widespread in real life, e.g., in disease gene identification tasks, all known genes which can cause diseases are positive instances, but it does not mean other unlabeled genes are all negative instances. By resorting to the *positive-unlabeled learning* (PU learning) methods [10, 12, 18, 25, 28], we can identify the underlying positive instances getting mixed into unlabeled genes [22].

There has been a long line of research for PU learning. In the beginning, the two-step strategy [17, 19, 27] is the mainstream approaches, in which reliable negative instances are carefully chosen at first, and then an ordinary semi-supervised learning model can be applied. The performance of these methods heavily depends on the quality of the chosen negative instances. Soon after, modeling PU learning as cost-sensitive learning [6, 11, 12] becomes popular, which can be roughly classified into two categories. The first category treats unlabeled instances as negative ones but assigns lower weight than positive ones [16, 18]. However, tuning a reasonable weight is time-consuming, and the critical data distribution information are not put into good use. The second category [10, 15] estimates the class prior of unlabeled data first [2, 7, 24] and then uses this prior to design some unbiased loss functions [10], followed by a cost-sensitive learning method. These methods require accurate class prior estimation and well-designed loss functions, which are too strict to be met in most cases.

This paper proposes a novel instance-dependent weighting method *entropy weight allocation* (EWA) for PU learning. Specifically, we first make an *optimal transport* (OT) from unlabeled instances to labeled instances [8] and define a novel distance measure for each unlabeled instance by OT coefficient distribution. Based on this distance, we allocate each unlabeled instance a weight indicating the possibility that it is an underlying negative instance. The greater the distance, the larger the weight, and hence the more likely to be a true negative instance. With these weights, any ordinary weighted learning models can be directly applied to obtain a PU classifier. Since all the unlabeled instances are taken into consideration and allocated a self-dependent weight, EWA successfully avoids the drawbacks of existing PU learning methods.

Notice that the original OT is linear programming, which encourages a sparse OT coefficient distribution and makes the underlying positive instances hardly distinguished. We overcome this difficulty by introducing an entropy regularization for OT, which will produce an almost uniform OT coefficient distribution for underlying negative instances while keep the OT coefficient distribution sparse for underlying positive instances. We also derive a generalization error bound by analyzing the Rademacher complexity when weighted learning models are kernel methods. By concatenating EWA with four celebrated classification models, i.e., *support vector machine* (SVM) [26], *optimal margin distribution machine* (ODM) [29], *gradient boosting decision tree* (GBDT) [5], and *neural network* (NN) [23], we show

*National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, 430074, China. {guwenly, tengzhang, hjin}@hust.edu.cn

that EWA is a broad-spectrum weighting method that can boost almost all the mainstream machine learning models for PU learning. The main contributions of this paper are summarized as below:

- We propose a novel distance measure based on optimal transport with high separability for underlying positive and negative instances.
- We derive the generalization error bound by analyzing the Rademacher complexity when weighted learning models are kernel methods.
- We empirically concatenate EWA with four mainstream classification models and verify that it is a broad-spectrum weighting method.

The rest of the paper is organized as follows. Section 2 is preliminary. Section 3 describes the proposed method EWA, and the experimental results are illustrated in Section 4. In the end, we conclude the paper in Section 5.

2 Preliminary

Throughout the paper, scalars are denoted by normal letters (e.g., y), while vectors and matrices are denoted by boldface lower and upper case letters respectively (e.g., $\boldsymbol{x}$ and $\mathbf{X}$). Sets are designated by upper case letters with mathcal font (e.g., $\mathcal{S}$). In particular, $[m]$ is the integer set $\{1, 2, \ldots, m\}$. For any positive integer d, the d-dimensional all-one vector and simplex are denoted by $\mathbf{1}_d$ and Δ_d respectively. For any matrices $\mathbf{X}$ and $\mathbf{Y}$ with the same size, the inner product is defined as $\langle \mathbf{X}, \mathbf{Y} \rangle \triangleq \sum_{i,j} X_{ij} Y_{ij}$. For any vector $\boldsymbol{\alpha}$, $\mathrm{diag}(\boldsymbol{\alpha})$ denotes the diagonal matrix with main diagonal equal to $\boldsymbol{\alpha}$.

2.1 PU Learning The training set of size m is m triples drawn i.i.d. from some unknown distribution over $\mathcal{X} \times \mathcal{Y} \times \mathcal{O}$, where $\mathcal{X}$ is the feature space, $\mathcal{Y} = \{1, -1\}$ is the label space, $\mathcal{O} = \{0, 1\}$ in which the boolean variable indicates whether the instance is labeled. Without loss of generality, we assume the first p instances $\mathcal{P} = \{\boldsymbol{x}_i\}_{i \in [p]}$ are positive, i.e., $y_i = o_i = 1$ for any $i \in [p]$, while the rest $u = m - p$ instances $\mathcal{U} = \{\boldsymbol{z}_j\}_{j \in [u]}$ are unlabeled, i.e., $o_i = 0$ for any $i \in [m] \setminus [p]$ and the corresponding y_i is ambiguous and can be either 1 or -1. The class prior of $\mathcal{U}$ is denoted by $\pi = \mathbb{P}(y = 1 | o = 0)$.

The goal of PU learning is the same as the standard binary classification, training a model to distinguish between positive and negative instances. This paper follows the common *s̲elected c̲ompletely a̲t r̲andom* (SCAR) assumption of PU learning: the observed positive instances are selected completely at random from all positive instances.

2.2 Optimal Transport Let $\boldsymbol{u} \in \Delta_u$ and $\boldsymbol{p} \in \Delta_p$ represent two discrete probability distributions, respectively. The set of all admissible couplings $\mathcal{C}(\boldsymbol{u}, \boldsymbol{p})$ is given by

$$\mathcal{C}(\boldsymbol{u}, \boldsymbol{p}) = \{\mathbf{T} \in \mathbb{R}_+^{u \times p} \mid \mathbf{T}\mathbf{1}_p = \boldsymbol{u},\ \mathbf{T}^\top \mathbf{1}_u = \boldsymbol{p}\}$$

where $\mathbf{T}$ is the OT coefficient matrix with the (j, i)-th entry describing the amount of mass flow transported from u_j toward p_i. Given a cost matrix $\mathbf{C} \in \mathbb{R}^{u \times p}$, OT minimizes the transporting cost from $\boldsymbol{u}$ to $\boldsymbol{p}$:

$$\min_{\mathbf{T} \in \mathcal{C}(\boldsymbol{u}, \boldsymbol{p})} \langle \mathbf{C}, \mathbf{T} \rangle = \min_{\mathbf{T} \in \mathcal{C}(\boldsymbol{u}, \boldsymbol{p})} \sum_{j \in [u]} \sum_{i \in [p]} C_{ij} T_{ij}$$

The above problem is an ordinary linear programming problem [13]. In this paper, we adopt the entropy regularized version [8]:

$$\mathrm{OT}_\lambda(\boldsymbol{u}, \boldsymbol{p}) = \min_{\mathbf{T} \in \mathcal{C}(\boldsymbol{u}, \boldsymbol{p})} \langle \mathbf{C}, \mathbf{T} \rangle - \lambda \cdot H(\mathbf{T})$$

where $H(\mathbf{T}) = -\sum_{ij} T_{ij} (\log T_{ij} - 1)$ and λ is a trading-off parameter. This form encourages a uniform transport and makes it easier to distinguish underlying positive instances from unlabeled data. We will detail this in the next section.

3 Proposed Method

In this section, we present our EWA method. Specifically, we first solve the entropy regularized OT by the Sinkhorn-Knopp algorithm. Then we calculate an entropy distance based on the obtained OT coefficients and normalize it as the weight for each unlabeled instance, indicating the possibility of being an underlying negative instance. With this weighted dataset, we can apply any ordinary weighted learning model. Finally, we derive a Rademacher generalization bound when models are kernel methods.

3.1 Entropy Distance We make an optimal transport from unlabeled instances to labeled instances. The original OT is linear programming, which encourages a sparse OT coefficient distribution and makes it difficult to distinguish the underlying positive instances. By introducing the entropy regularization, we can get a smoother distribution. More specifically, we treat each instance equally, i.e., assigning a probability value $1/u$ to each unlabeled instance while $1/p$ to each positive instance. Then we construct the cost matrix $\mathbf{C}$ by calculating the Euclidean distance between the instances and solve the entropy regularized OT:

$$\begin{aligned} \mathrm{OT}_\lambda(\boldsymbol{u}, \boldsymbol{p}) &= \min_{\mathbf{T} \in \mathcal{C}(\boldsymbol{u}, \boldsymbol{p})} \langle \mathbf{C}, \mathbf{T} \rangle - \lambda \cdot H(\mathbf{T}) \\ \text{s.t. } \mathbf{T}\mathbf{1}_p &= \frac{\mathbf{1}_u}{u},\ \mathbf{T}^\top \mathbf{1}_u = \frac{\mathbf{1}_p}{p} \end{aligned} \tag{3.1}$$

where $C_{ij} = \|\boldsymbol{z}_j - \boldsymbol{x}_i\|_2$ is the Euclidean distance between $\boldsymbol{z}_j$ and $\boldsymbol{x}_i$.

We solve the Equation 3.1 by the Sinkhorn-Knopp algorithm [8]. The Lagrangian of Equation 3.1 with dual variables $\boldsymbol{\gamma} \in \mathbb{R}^u$, $\boldsymbol{\zeta} \in \mathbb{R}^p$ is

$$L(\mathbf{T}, \boldsymbol{\gamma}, \boldsymbol{\zeta}) = \sum_{j \in [u]} \sum_{i \in [p]} (T_{ij} C_{ij} + \lambda T_{ij} (\log T_{ij} - 1)) + \boldsymbol{\gamma}^\top \left(\mathbf{T} \mathbf{1}_p - \frac{\mathbf{1}_u}{u} \right) + \boldsymbol{\zeta}^\top \left(\mathbf{T}^\top \mathbf{1}_u - \frac{\mathbf{1}_p}{p} \right)$$

Setting the partial derivative of T_{ij} to zero yields

$$\frac{\partial L}{\partial T_{ij}} = 0 \Longrightarrow T_{ij} = \exp\left(\frac{\gamma_j}{\lambda}\right) \exp\left(-\frac{C_{ij}}{\lambda}\right) \exp\left(\frac{\zeta_i}{\lambda}\right) \Longrightarrow \mathbf{T} = \operatorname{diag}(\boldsymbol{\alpha}) \mathbf{K} \operatorname{diag}(\boldsymbol{\beta})$$

where $\boldsymbol{\alpha} = \exp(\boldsymbol{\gamma}/\lambda)$, $\mathbf{K} = \exp(-\mathbf{C}/\lambda)$, and $\boldsymbol{\beta} = \exp(\boldsymbol{\zeta}/\lambda)$ are the element-wise exponential of $\boldsymbol{\gamma}/\lambda$, $-\mathbf{C}/\lambda$, and $\boldsymbol{\zeta}/\lambda$ respectively. The Sinkhorn-Knopp algorithm iteratively updates the variables $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ to make the row and column marginals of $\mathbf{T}$ equal their target values:

$$\boldsymbol{\alpha} = \boldsymbol{u} \oslash (\mathbf{K} \boldsymbol{\beta}), \quad \boldsymbol{\beta} = \boldsymbol{p} \oslash (\mathbf{K}^\top \boldsymbol{\alpha})$$

where $\oslash$ is the element-wise division.

The heat maps of the OT coefficient matrix of mushroom and mnist datasets are shown in Figure 1. We can find that the OT coefficients are almost uniform for underlying negative instances while sparse for underlying positive instances. Based on this observation, we view the OT coefficients as a probability distribution and define an entropy distance from each unlabeled instance to the positive instances.

Definition 3.1. (entropy distance) *Given the OT coefficient matrix* $\mathbf{T}$, *the entropy distance from an unlabeled instance to the positive instances is defined as*

$$d(\boldsymbol{z}_j, \mathcal{P}) = -\sum_{i \in [p]} (u \cdot T_{ji}) \log(u \cdot T_{ji})$$

Since the sum of each row of $\mathbf{T}$ is only $1/u$, we first normalize it as a distribution by multiplying T_{ij} with the number of unlabeled instances u. Notice that ordinary entropy is a concave function achieving the maximum at uniform distribution, this entropy distance exactly expresses that the underlying negative instances are farther away from the positive instances.

Besides the entropy regularization, the cost matrix also plays an important role. It is supposed to be able to distinguish the positive and negative instances well. In this paper, we use the Euclidean distance as cost; however, a better cost matrix can be constructed by *metric learning* [3, 9], which is left for future works. Algorithm 1 summarizes the pseudo-code of how to calculate the entropy distance.

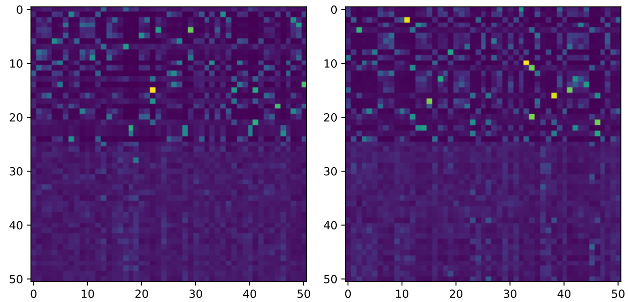

Figure 1: The heat maps of the OT coefficient matrix $\mathbf{T}$ transported from 50 unlabeled instances (half positive and half negative) to 50 positive instances from mushroom (left) and mnist (right) datasets. Each row represents the OT coefficients for an unlabeled instance. It can be seen that the first half underlying positive instances have a sparse OT coefficients while the second half underlying negative instances have an almost uniform OT coefficients.

Algorithm 1 Entropy Distance

Input: Probability $\boldsymbol{u} \in \Delta_u$, $\boldsymbol{p} \in \Delta_p$, cost matrix $\mathbf{C} \in \mathbb{R}_+^{u \times p}$, regularization parameter $\lambda > 0$, tolerance $\epsilon > 0$, $err = inf$.
Initialize $k \leftarrow 0$
Let $\boldsymbol{\alpha}^0 \leftarrow \mathbf{1}_u/u$, $\boldsymbol{\beta}^0 \leftarrow \mathbf{1}_p/p$
$\mathbf{K} \leftarrow e^{-\mathbf{C}/\lambda}$
$\mathbf{T}^0 \leftarrow \operatorname{diag}(\boldsymbol{\alpha}^0) \mathbf{K}^\top \operatorname{diag}(\boldsymbol{\beta}^0)$
while $err > \epsilon$ **do**
  $k \leftarrow k + 1$
  $\boldsymbol{\alpha}^k \leftarrow \boldsymbol{u} \oslash (\mathbf{K} \boldsymbol{\beta}^{k-1})$
  $\boldsymbol{\beta}^k \leftarrow \boldsymbol{p} \oslash (\mathbf{K}^\top \boldsymbol{\alpha}^{k-1})$
  $\mathbf{T}^k = \operatorname{diag}(\boldsymbol{\alpha}^k) \mathbf{K} \operatorname{diag}(\boldsymbol{\beta}^k)$
  $err = \langle \mathbf{T}^k, \mathbf{C} \rangle - \langle \mathbf{T}^{k-1}, \mathbf{C} \rangle$
end while
for $j \leftarrow 1$ to u **do**
  $d(\boldsymbol{z}_j, \mathcal{P}) = -\sum_{i \in [p]} (u \cdot T_{ji}) \log(u \cdot T_{ji})$
end for
Output: Entropy distance vector.

3.2 Entropy Weight Allocation Once obtaining the entropy distance, we can calculate the instance-dependent weight for each unlabeled instance.

Definition 3.2. (Entropy Weight) *Given the entropy distance, the instance-dependent entropy weight for each unlabeled instance is calculated by min-max normalizing the entropy distance*

$$\mathrm{EW}_j = \frac{d(\boldsymbol{z}_j, \mathcal{P}) - \min_l d(\boldsymbol{z}_l, \mathcal{P})}{\max_l d(\boldsymbol{z}_l, \mathcal{P}) - \min_l d(\boldsymbol{z}_l, \mathcal{P})}$$

In other words, we treat each unlabeled instance as a negative instance with weight $\mathrm{EW}_j \leq 1$. As for the positive instances, their weights are all fixed as 1. Then the PU learning problem is transformed into a weighted learning problem. We concatenate EWA with four mainstream learning models which can handle instance-dependent weights, i.e., SVM, ODM, NN, and GBDT.

For SVM, ODM, and NN, their EWA-concatenated versions (EWSVM, EWODM, EWNN) can be unified written as

$$\min_{\boldsymbol{w}} \ \frac{1}{2}\|\boldsymbol{w}\|_2^2 + C \sum_{i\in[m]} l(y_i, f(\boldsymbol{x}_i)) \cdot \mathrm{EW}_i^{1_{i\notin[p]}}$$

For SVM, $l(y_i, f(\boldsymbol{x}_i)) = \max(0, 1 - y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b))$ is the hinge loss. For ODM, $l(y_i, f(\boldsymbol{x}_i)) = (\xi_i^2 + \mu\epsilon_i^2)/(1-\theta)^2$ where $\xi_i = \max(0, 1-\theta - y_i\boldsymbol{w}^\top \boldsymbol{x}_i)$ and $\epsilon_i = \max(0, y_i\boldsymbol{w}^\top \boldsymbol{x}_i - 1 - \theta)$. For NN, $l(y_i, f(\boldsymbol{x}_i)) = y_i \log(f(\boldsymbol{x}_i)) + (1-y_i)\log(1-f(\boldsymbol{x}_i))$ is the cross-entropy loss where f is the multi-layer perceptron.

For GBDT, the EWGBDT can be written as

$$\min\{y_i\phi(f(\boldsymbol{x}_i)) + (1-y_i)[f(\boldsymbol{x}_i) + \phi(f(\boldsymbol{x}_i)]\} \cdot \mathrm{EW}_i^{1_{i\notin[p]}}$$

where $\phi(f(\boldsymbol{x}_i)) = \log(1 + e^{-f(\boldsymbol{x}_i)})$, $f(\boldsymbol{x}) = \sum_{j\in[t]} f_j(\boldsymbol{x})$, and t is the number of decision trees participating in the boosting.

Notice that the first p instances are positive, therefore $\mathrm{EW}_i^{1_{i\notin[p]}}$ means the entropy weights are multiplied to each unlabeled instance. With these weights, any ordinary weighted learning models can be directly applied to obtain a PU classifier.

3.3 Generalization Bound

In this section, we give the generalization bound of EWSVM. We first introduce the following function:

$$\ell^\rho(x) = \begin{cases} 0 & \text{if } x \geq \rho \\ 1 - {}^{x}/_{\rho} & \text{if } 0 \leq x \leq \rho \\ 1 & \text{otherwise} \end{cases}$$

where ρ is the margin as defined in [21]. To derive the main theorem, we need the following two lemmas.

Lemma 3.1. (Generalization Bound [1]) *Let $\mathcal{F}$ be a $[0,1]$-valued function class on $\mathcal{X}$ and $f \in \mathcal{F}$. Given dataset $\mathcal{S} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m\}$ i.i.d. drawn from $\mathcal{X}$, then for any $\delta > 0$, with probability at least $1-\delta$, we have*

$$\sup_{f\in\mathcal{F}} \left(\mathbb{E}f(\boldsymbol{x}) - \frac{1}{m}\sum_{i\in[m]} f(\boldsymbol{x}_i) \right) \leq 2\mathcal{R}_{\mathcal{S}}(\mathcal{F}) + \sqrt{\frac{\ln(1/\delta)}{2m}}$$

Lemma 3.2. (Talagrand Contraction Lemma [1]) *If the function $l : \mathbb{R} \to \mathbb{R}$ is Lipschitz continuous with constant L, then*

$$\mathcal{R}_{\mathcal{S}}(l \circ \mathcal{F}) \leq L\mathcal{R}_{\mathcal{S}}(\mathcal{F})$$

where $l \circ \mathcal{F}$ represents the composition of l and all $f \in \mathcal{F}$.

Theorem 3.1. (Generalization Bound) *Assume $\mathcal{X}$ is a finite set, with $r = \max_{\boldsymbol{x}\in\mathcal{X}} \|\boldsymbol{x}\|_2$. Let p and u be the size of the positive instances and the unlabeled instances. C is the regularization hyperparameter and EW is the entropy weight. h is the classifier parameter learned by the algorithm. For any $\delta > 0$, with probability at least $1-\delta$, we have*

$$\mathcal{R}(h) - \hat{\mathcal{R}}(h) \leq \frac{2r\sqrt{2(1+\Phi)C}}{\rho\sqrt{pu}/(\sqrt{p}+\sqrt{u})} + \sqrt{\frac{\ln(1/\delta)}{2(p+u)}}$$

where $\Phi = \sum_{j\in[u]} {}^{\mathrm{EW}_j}/_{u}$.

Proof. According to lemma 3.1 and McDiarmid's inequality [20], we have $\mathcal{R}(h) \leq \hat{\mathcal{R}}(h) + 2\mathcal{R}_{\mathcal{S}}(h) + \sqrt{\frac{\ln(1/\delta)}{2m}}$. By scaling the inequality, we obtain $\|\boldsymbol{w}\|^2 \leq 2(1+\Phi)C$. Further with lemma 3.2 and Rademacher Complexity [1], we can derive the upper bound of the risk on positive instances and unlabeled instances, i.e., $\mathcal{R}_p(h)$ and $\mathcal{R}_u(h)$, thus the upper bound of $2\mathcal{R}_{\mathcal{S}}(h)$ can be obtained. □

4 Experiments

In this section, we concatenate EWA with four celebrated classification models, i.e., SVM, ODM, GBDT, and NN. We first describe twelve real-world datasets, five comparable methods, and experiment settings, then show the weights distribution of unlabeled instances. In addition to comparing with the state-of-the-art PU learning methods, we compare the performance difference between the original models and the EWA models to demonstrate the effectiveness of the EWA method. Moreover, we verify the robustness of the algorithm under class prior shift and different entropy regularization parameters. Finally, we model the malicious URLs detection as a PU learning task.

4.1 Datasets

We utilize twelve datasets from the UCI Machine Learning Repository, including mushroom, usps, shuttle, house, spambase, mnist, banknote, fashion, phoneme, PhishingWebsites, kr-vs-kp, musk. A dataset summary is listed in Table 1, including the number of positive, negative instances and feature dimensions. Some datasets have multiple classes, imitating the previous processing of [14], we treat it as a binary classification problem. The specific method is choosing one of the classes as the positive class, and the other classes as negative class.

Table 1: Details of the datasets in our experiments

Dataset	#Ins.	#Fea.	#Pos.	#Neg.
mushroom	8124	23	3916	4208
usps	9298	257	1553	7745
shuttle	58000	10	45586	12414
house	20640	9	8914	11726
spambase	4601	58	1813	2788
mnist	60000	780	6742	53258
banknote	1372	5	762	610
fashion	70000	785	7000	63000
phoneme	5404	6	3818	1586
PhishingWebsites	51520	31	6157	4898
kr-vs-kp	3196	37	1669	1527
musk	6598	178	1017	5581

4.2 Compared Methods To demonstrate the superiority of EWA, we compare it with Spy [19], WLR [16], EN [12], nnPU [15], and PW [4]. Considering that class prior estimation is not used in nnPU and PW, and the original value is used directly, so we use KM [24] to estimate the class prior.

4.3 Setup For the training dataset, we select p instances as the $\mathcal{P}$ dataset randomly. Let class prior π denote the proportion of positive instances in $\mathcal{U}$, we select $\pi \cdot u$ positive instances and $(1-\pi)\cdot u$ negative instances randomly to form an unlabeled dataset. The values of p and u apply [14]. Except that the banknote and musk datasets have fewer instances, p and u are 200 and 400 respectively, p and u in the other datasets are 400 and 800 respectively. In order to demonstrate different scenarios, we set $\pi = \{0.3, 0.5, 0.7\}$.

4.4 Entropy Weight Figure 2 displays the entropy weights distribution of mushroom and mnist. For the convenience of observation, let $\pi = 0.5$. Observing the distribution of weights, we find that the weights of positive instances are almost all less than 0.5, and the weights of negative instances are almost all greater than 0.5. A small number of instances do not match expectation. Overall, the EWA can be a good measure with high separability for underlying positive and negative instances.

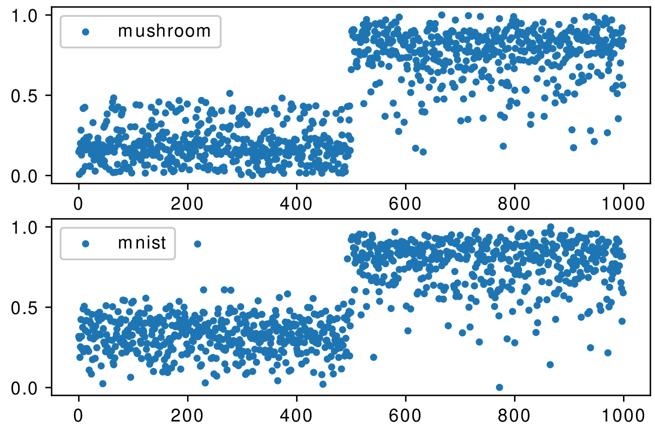

Figure 2: Unlabeled instances weights distribution

4.5 Performance Table 2 summarizes the performance of nine methods on twelve datasets in terms of F1 score over 20 trials. Table 3 counts the win/tie/loss for EWA methods and compared methods. From the results, we see that our proposed EWA methods outperform most of the baselines and achieve the best performance in all cases.

Analyzing the results in more detail, we find that 1) EWSVM, EWODM, EWGBDT, and EWNN achieve the best accuracy on different datasets, the performance is related to the trained model. 2) In some scenarios, such as phoneme, nnPU achieve very good performance. But compared with EWNN, considering their classification models are neural networks, the performance of EWNN is significantly better than nnPU.

4.6 Original Model and EWA Model In Table 4, we compare the original models with the EWA models, and find that: 1) The original models do not perform well for PU learning, but the performance has been greatly improved after the integration of EWA. 2) When the class prior is 0.3, the original models also achieve a high accuracy rate. However, as the class prior increases, the accuracy of all original models decays rapidly. So as to verify the performance of the algorithm in different scenarios, we need to set a larger class prior interval. 3) The EWA models have high robustness in different class prior scenarios. Even if the class prior increases, the accuracy decays little. In summary, EWA is a broad-spectrum weighting method that can boost almost all the mainstream machine learning models for PU learning.

4.7 Class Prior Shift We consider the scenario of class prior shift, which means, the class prior of the training dataset and the test dataset are different.

Table 2: Results on twelve datasets in terms of F1 score over 20 trials are reported. In each class prior scenario, the best result on each dataset is bolded. ●/○ indicates the performance of EWSVM/EWODM/EWGBDT/EWNN is significantly better/worse than the compared method (pairwise t-test at 0.05 significance level).

Dataset	π	Spy	WLR	EN	nnPU	PW	EWSVM	EWODM	EWGBDT	EWNN
mushroom	.3	.977±.016 ●●●●	.919±.014 ●●●●	.987±.005 ●●●●	.945±.015 ●●●●	.899±.022 ●●●●	.997±.002	.994±.005	.995±.002	**.998±.001**
	.5	.977±.013 ●●●●	.748±.012 ●●●●	.986±.005 ●●●●	.935±.013 ●●●●	.907±.027 ●●●●	.995±.004	.991±.007	.995±.003	**.996±.002**
	.7	.951±.022 ●●●●	.592±.017 ●●●●	.981±.006 ●●●●	.933±.015 ●●●●	.921±.033 ●●●●	.988±.008	.986±.005	.995±.002	**.996±.001**
usps	.3	.961±.009 ●●	.954±.003 ●●●●	.969±.005 ●●○○	.919±.019 ●●●●	.940±.027 ●●●●	**.987±.002**	.984±.003	.965±.005	.961±.003
	.5	.891±.069 ●●●●	.908±.006 ●●●●	.961±.010 ●●	.895±.020 ●●●●	.944±.017 ●●●●	**.983±.003**	.981±.004	.963±.008	.963±.002
	.7	.867±.060 ●●●●	.864±.005 ●●●●	.925±.016 ●●●●	.856±.036 ●●●●	.944±.030 ●● ●	.973±.006	**.977±.008**	.959±.008	.963±.004
shuttle	.3	.865±.031 ●●	.551±.011 ●●●●	.926±.004 ○○●●	.915±.089 ●	.855±.054 ●●●●	.884±.020	.885±.015	**.993±.006**	.951±.005
	.5	.877±.052 ●●	.229±.017 ●●●●	.914±.005 ○○●●	.966±.006 ○○ ○	.883±.067 ●●	.877±.025	.881±.007	**.972±.022**	.948±.005
	.7	.857±.023 ●●●●	.209±.000 ●●●●	.882±.007 ● ●●	.896±.036 ●●	.897±.069 ●●	.907±.040	.883±.014	**.955±.026**	.949±.004
house	.3	.953±.012 ●●●	.865±.007 ●●●●	.965±.001 ● ●○	.808±.211 ●●●●	.925±.028 ●●●●	.967±.001	.965±.003	**.971±.004**	.958±.002
	.5	.803±.150 ●● ●	.688±.011 ●●●●	.963±.002 ●● ○	.941±.019 ●● ●	.898±.026 ●● ●	**.967±.001**	.958±.014	.883±.191	.958±.001
	.7	.571±.175 ●●●●	.603±.004 ●●●●	.950±.018 ●	.865±.142 ●● ●	.906±.024 ●● ●	**.968±.002**	.951±.004	.906±.159	.956±.001
spambase	.3	.526±.031 ●●●●	.732±.010 ●●●●	.728±.038 ●●●●	.803±.016 ●●●●	.796±.014 ●●●●	.847±.003	.851±.004	**.915±.014**	.877±.005
	.5	.540±.127 ●●●●	.657±.010 ●●●●	.607±.033 ●●●●	.837±.012 ● ●	.753±.026 ●●●●	.844±.007	.839±.015	.873±.087	**.881±.004**
	.7	.427±.033 ●●●●	.622±.004 ●●●●	.501±.019 ●●●●	.839±.015 ● ●	.690±.115 ●●●●	.841±.009	.833±.008	.836±.150	**.876±.006**
mnist	.3	.938±.031 ●●●	.965±.004 ●●●○	.969±.005 ●●●○	.857±.022 ●●●●	.974±.011 ●● ○	.980±.002	**.987±.004**	.977±.002	.948±.003
	.5	.923±.007 ●●●●	.932±.004 ●●●●	.956±.008 ●●●○	.810±.036 ●●●●	.948±.024 ●●●	.977±.004	**.985±.006**	.977±.003	.950±.004
	.7	.911±.012 ●●●●	.905±.005 ●●●●	.934±.008 ●●●●	.767±.043 ●●●●	.954±.019 ●●●	.966±.007	**.981±.006**	.973±.005	.950±.003
banknote	.3	.959±.015 ●●●○	.667±.011 ●●●●	.957±.010 ●●●○	.978±.017 ○	.900±.073 ●●●●	**.978±.002**	.976±.011	.975±.010	.943±.012
	.5	.883±.105 ●●●●	.508±.018 ●●●●	.951±.013 ●●●	.977±.014 ○○	.865±.096 ●●●●	**.978±.002**	.974±.007	.962±.018	.947±.011
	.7	.819±.125 ●●●●	.410±.013 ●●●●	.821±.059 ●●●●	.937±.087 ●	.830±.119 ●●●●	**.982±.006**	.975±.024	.929±.029	.948±.012
fashion	.3	.979±.002 ●●	.866±.011 ●●●●	.950±.007 ●● ●	.938±.019 ●● ●	.929±.023 ●● ●	.984±.001	**.986±.003**	.953±.103	.976±.001
	.5	.924±.054 ●● ●	.730±.012 ●●●●	.935±.006 ●● ●	.913±.030 ●● ●	.904±.030 ●● ●	.980±.004	**.985±.005**	.950±.104	.982±.001
	.7	.937±.028 ●● ●	.626±.012 ●●●●	.919±.011 ●● ●	.897±.024 ●● ●	.911±.035 ●● ●	.970±.008	**.982±.008**	.897±.165	.978±.001
phoneme	.3	.696±.000 ●●●●	.531±.025 ●●●●	.752±.009 ●○●●	.693±.043 ●●●●	.827±.011 ○○○●	.772±.005	.755±.019	.792±.010	**.837±.003**
	.5	.696±.001 ●●●●	.358±.030 ●●●●	.723±.012 ●●●●	.807±.033 ○○○●	.743±.029 ● ●●	.776±.009	.761±.017	.789±.009	**.838±.005**
	.7	.695±.000 ●●●●	.303±.001 ●●●●	.704±.008 ●●●●	.829±.002 ○○○●	.723±.070 ●●	.756±.024	.754±.011	.773±.018	**.836±.003**
PhishingW.	.3	.833±.035 ●●●●	.776±.028 ●●●●	.717±.010 ●●●●	.875±.004 ●○●●	.783±.019 ●●●●	.889±.007	.871±.007	**.912±.005**	.885±.004
	.5	.776±.058 ●●●●	.827±.023 ●●●●	.618±.011 ●●●●	.855±.010 ●●●●	.666±.043 ●●●●	.873±.010	.854±.012	**.908±.004**	.886±.007
	.7	.651±.051 ●●●●	.844±.014 ●● ●	.524±.012 ●●●●	.800±.022 ● ●●	.628±.100 ●●●●	.845±.018	.847±.019	**.902±.011**	.886±.004
kr-vs-kp	.3	.906±.026 ●●●●	.746±.015 ●●●●	.915±.009 ●●●●	.784±.031 ●●●●	.807±.011 ●●●●	**.945±.003**	.923±.007	.929±.007	.928±.005
	.5	.862±.078 ●●●●	.611±.013 ●●●●	.882±.018 ●●●○	.802±.036 ●●●●	.753±.007 ●●●●	**.941±.009**	.920±.012	.908±.013	.929±.008
	.7	.789±.082 ●●●●	.547±.010 ●●●●	.791±.028 ●●●●	.852±.039 ● ●	.593±.150 ●●●●	**.928±.013**	.891±.023	.891±.021	.859±.010
musk	.3	.938±.037 ●●●	.951±.005 ●●●○	.985±.005 ○ ●○	.881±.041 ●●●●	.699±.089 ●●●●	.977±.007	.983±.007	**.997±.002**	.962±.011
	.5	.907±.043 ●●●	.903±.004 ●●●●	.971±.011 ●○	.904±.049 ●●●●	.633±.061 ●●●●	.963±.017	.954±.005	**.992±.003**	.931±.007
	.7	.792±.079 ●●●●	.869±.005 ●●●●	.927±.023 ● ●●	.834±.045 ●●●●	.620±.054 ●●●●	.911±.024	.918±.014	**.980±.008**	.875±.013

The class prior value set of the training dataset is still $\{0.3, 0.5, 0.7\}$, while the test dataset considers the arithmetic set from 0.1 to 0.9. The test datasets are mushroom, usps, fashion, and PhishingWebsites.

Table 3: The win/tie/loss counts for EWA methods and compared methods.

Methods	Spy	WLR	EN	nnPU	PW
EWSVM	34/2/0	36/0/0	32/1/3	28/5/3	32/3/1
EWODM	34/2/0	36/0/0	27/6/3	25/7/4	31/4/1
EWGBDT	31/5/0	35/1/0	29/6/1	23/10/3	28/7/1
EWNN	29/6/1	34/0/2	24/3/9	30/3/3	33/2/1

From Figure 3, we can see that the EWA methods still have high robustness in the class prior shift scenario. When $\pi = 0.5$, the model accuracy rate changes most smoothly, which proves that when the instance distribution in the training dataset is balanced, it is slightly affected by the change in the positive and negative instances distribution of the test dataset. When $\pi = 0.3$ and 0.7, the performance change is more dramatic, which indicates that the ratio of positive and negative instances will affect the decision boundary of the model. But overall, the results in different scenarios are similar, the model is robust.

4.8 Parameter Analysis We need to discuss the hyperparameter λ. If λ is too small, the model degenerates to the original optimal transport. While λ is too large, the OT coefficient is almost uniformly distributed.

Table 4: The F1 score of the original models and the corresponding EWA models over 20 trials. •/○ indicates the performance of the EWA model is significantly better/worse than the original model (pairwise t-test at 0.05 significance level). The win/tie/loss counts are summarized in the last row.

Dataset	π	SVM	EWSVM	ODM	EWODM	GBDT	EWGBDT	NN	EWNN
mushroom	.3	.980±.016 •	.997±.002	.921±.024 •	.994±.005	.921±.028 •	.995±.002	.878±.025 •	.998±.001
	.5	.727±.019 •	.995±.004	.764±.015 •	.991±.007	.629±.040 •	.995±.003	.645±.086 •	.996±.002
	.7	.526±.014 •	.988±.008	.553±.023 •	.986±.005	.531±.004 •	.995±.002	.390±.106 •	.996±.001
usps	.3	.987±.002	.987±.002	.958±.011 •	.984±.003	.949±.007 •	.965±.005	.894±.038 •	.961±.003
	.5	.897±.016 •	.983±.003	.903±.006 •	.981±.004	.890±.009 •	.963±.008	.633±.037 •	.963±.002
	.7	.830±.002 •	.973±.006	.841±.008 •	.977±.008	.840±.003 •	.959±.008	.264±.053 •	.963±.004
shuttle	.3	.771±.008 •	.884±.020	.785±.015 •	.885±.015	.852±.021 •	.993±.006	.752±.049 •	.951±.005
	.5	.240±.106 •	.877±.025	.331±.028 •	.881±.007	.435±.028 •	.972±.022	.443±.123 •	.948±.005
	.7	.216±.000 •	.907±.040	.251±.005 •	.883±.014	.250±.029 •	.955±.026	.053±.058 •	.949±.004
house	.3	.956±.005 •	.967±.001	.877±.023 •	.965±.003	.947±.016 •	.971±.004	.838±.016 •	.958±.002
	.5	.654±.096 •	.967±.001	.712±.018 •	.958±.014	.733±.045	.883±.191	.505±.036 •	.958±.001
	.7	.573±.000 •	.968±.002	.612±.008 •	.951±.004	.577±.016 •	.906±.159	.178±.026 •	.956±.001
spambase	.3	.671±.023 •	.847±.003	.742±.021 •	.851±.004	.812±.010 •	.915±.014	.671±.026 •	.877±.005
	.5	.617±.001 •	.844±.007	.632±.027 •	.839±.015	.705±.010 •	.873±.087	.451±.027 •	.881±.004
	.7	.617±.000 •	.841±.009	.611±.012 •	.833±.008	.621±.006 •	.836±.150	.207±.047 •	.876±.006
mnist	.3	.986±.001 ○	.980±.002	.959±.006 •	.987±.004	.976±.003	.977±.002	.826±.017 •	.948±.003
	.5	.926±.009 •	.977±.004	.929±.013 •	.985±.006	.936±.006 •	.977±.003	.618±.069 •	.950±.004
	.7	.885±.000 •	.966±.007	.892±.008 •	.981±.006	.892±.002 •	.973±.005	.445±.147 •	.950±.003
banknote	.3	.961±.005 •	.978±.002	.755±.011 •	.976±.011	.777±.035 •	.975±.010	.741±.076 •	.943±.012
	.5	.615±.141 •	.978±.002	.557±.017 •	.974±.007	.581±.044 •	.962±.018	.366±.124 •	.947±.011
	.7	.430±.000 •	.982±.006	.445±.024 •	.975±.024	.456±.019 •	.929±.029	.077±.069 •	.948±.012
fashion	.3	.967±.007 •	.984±.001	.883±.014 •	.986±.003	.889±.010	.953±.103	.960±.009 •	.976±.001
	.5	.730±.044 •	.980±.004	.754±.011 •	.985±.005	.681±.018 •	.950±.104	.732±.144 •	.982±.001
	.7	.501±.002 •	.970±.008	.644±.018 •	.982±.008	.534±.007 •	.897±.165	.095±.084 •	.978±.001
phoneme	.3	.511±.101 •	.772±.005	.561±.024 •	.755±.019	.610±.013 •	.792±.010	.556±.079 •	.837±.003
	.5	.303±.014 •	.776±.009	.337±.029 •	.761±.017	.439±.056 •	.789±.009	.301±.077 •	.838±.005
	.7	.297±.000 •	.756±.024	.303±.008 •	.754±.011	.312±.006 •	.773±.018	.011±.021 •	.836±.003
PhishingW.	.3	.767±.017 •	.889±.007	.724±.008 •	.871±.007	.804±.022 •	.912±.005	.721±.020 •	.885±.004
	.5	.574±.032 •	.873±.010	.633±.014 •	.854±.012	.571±.025 •	.908±.004	.533±.029 •	.886±.007
	.7	.464±.010 •	.845±.018	.552±.013 •	.847±.019	.457±.012 •	.902±.011	.300±.043 •	.886±.004
kr-vs-kp	.3	.858±.014 •	.945±.003	.772±.027 •	.923±.007	.643±.012 •	.926±.010	.740±.025 •	.928±.005
	.5	.621±.028 •	.941±.009	.636±.026 •	.920±.012	.542±.014 •	.908±.013	.529±.041 •	.929±.008
	.7	.484±.015 •	.928±.013	.574±.029 •	.891±.023	.519±.009 •	.891±.021	.340±.056 •	.859±.010
musk	.3	.974±.009	.977±.007	.947±.016 •	.983±.007	.982±.004 •	.997±.002	.796±.090 •	.962±.011
	.5	.867±.022 •	.963±.017	.902±.011 •	.954±.005	.908±.006 •	.992±.003	.632±.132 •	.931±.007
	.7	.841±.001 •	.911±.024	.881±.005 •	.918±.014	.866±.004 •	.980±.008	.444±.161 •	.875±.013
EWA: w/t/l		33/2/1		36/0/0		33/3/0		36/0/0	

From the experimental point of view, a reasonable range of λ is between 0.01 and 0.1.

Figure 4 displays the performance of EWSVM under different class priors π and hyperparameter λ. The class prior value set is still $\{0.3, 0.5, 0.7\}$, and the test datasets are mushroom, usps, fashion, and PhishingWebsites. Observing the results, we get the following conclusions: Under reasonable entropy regularization, EWA has no strong sensitivity to hyperparameter λ.

4.9 Malicious URLs Detection With the development of the Internet, more and more URL attacks pose a huge threat to network security. Traditional malicious URL identification is mainly based on the blacklist and rule list, but these methods are not scalable and cannot cope with all attacks, and they cannot identify potential

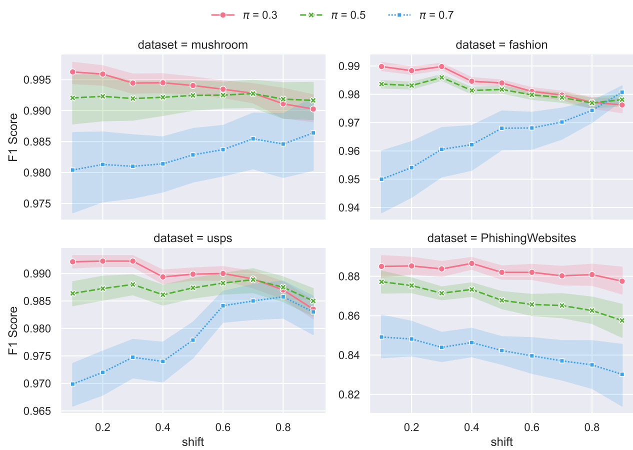

Figure 3: Class prior shift on four datasets. We set different test dataset class priors to verify the robustness of EWA in different scenarios. The horizontal axis represents the class prior of the test dataset.

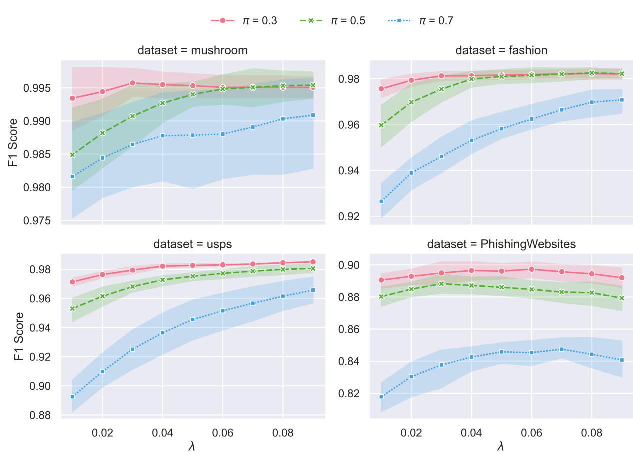

Figure 4: Parameter analysis on four datasets. We adjust the value of the entropy regularization parameter λ to verify that the algorithm is not sensitive to the parameter.

attacks and discover possible malicious URLs [30].

We model the malicious URLs detection as a PU learning task. In order to verify the effectiveness of EWA in the detection of malicious URLs, we used a UCI public dataset PhishingWebsites and a dataset obtained by feature extraction from the real URL dataset. Figure 5 shows the F1 score of different algorithms on two website datasets. As far as these two datasets are concerned, EWGBDT achieve the best accuracy.

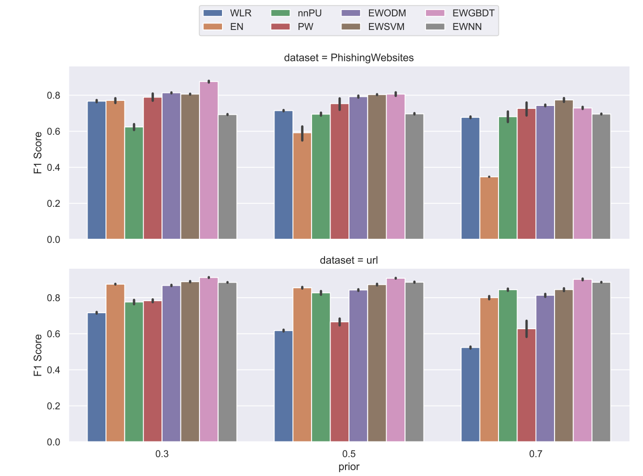

Figure 5: Malicious URLs detections. The F1 score of various PU learning methods on (a) PhishingWebsites and (b) url.

5 Conclusion

In this paper, we propose a novel instance-dependent weighting method EWA for PU learning by optimal transport. We make an optimal transport from unlabeled instances to labeled instances, use the entropy of the OT coefficient distribution to measure the probability that an unlabeled instance is an underlying negative instance, and concatenate the EWA with four celebrated classification models. We theoretically analyzed generalization error bound when learning models are kernel methods and performed extensive experiments to verify it is a broad-spectrum weighting method.

6 Acknowledgments

This work was supported by NSFC (62006088).

References

[1] P. L. Bartlett and S. Mendelson, *Rademacher and Gaussian complexities: Risk bounds and structural results*, Journal of Machine Learning Research, 3 (2002), pp. 463–482.

[2] J. Bekker and J. Davis, *Estimating the class prior in positive and unlabeled data through decision tree induction*, in Proceedings of the 32nd AAAI Conference on Artificial Intelligence, New Orleans, LA, 2018, pp. 2712–2719.

[3] K. Brian, *Metric learning: A survey*, Foundations and Trends in Machine Learning, 5 (2012), pp. 287–364.

[4] L. Chapel, M. Z. Alaya, and G. Gasso, *Partial optimal transport with applications on positive-unlabeled learning*, in Advances in Neural Information Processing Systems, Virtual, 2020, pp. 2903–2913.

[5] T. Chen and C. Guestrin, *Xgboost: A scalable tree boosting system*, in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794.

[6] X. Chen, C. Gong, and J. Yang, *Cost-sensitive positive and unlabeled learning*, Information Sciences, 558 (2021), pp. 229–245.

[7] M. Christoffel, G. Niu, and M. Sugiyama, *Class-prior estimation for learning from positive and unlabeled data*, in Proceedings of the 7th Asian Conference on Machine Learning, Hong Kong, China, 2015, pp. 221–236.

[8] M. Cuturi, *Sinkhorn distances: Lightspeed computation of optimal transport*, in Advances in Neural Information Processing Systems, Lake Tahoe, NV, 2013, pp. 2292–2300.

[9] J. V. Davis, B. Kulis, P. Jain, S. Sra, and I. S. Dhillon, *Information-theoretic metric learning*, in Proceedings of the 24th International Conference on Machine Learning, Corvallis, OR, 2007, pp. 209–216.

[10] M. Du Plessis, G. Niu, and M. Sugiyama, *Convex formulation for learning from positive and unlabeled data*, in Proceedings of the 32nd International Conference on Machine Learning, Lille, France, 2015, pp. 1386–1394.

[11] M. C. Du Plessis, G. Niu, and M. Sugiyama, *Analysis of learning from positive and unlabeled data*, in Advances in Neural Information Processing Systems, Montreal, Canada, 2014, pp. 703–711.

[12] C. Elkan and K. Noto, *Learning classifiers from only positive and unlabeled data*, in Proceedings of the 14th International Conference on Knowledge Discovery and Data Mining, Las Vegas, NV, 2008, pp. 213–220.

[13] L. Kantorovich, *On the transfer of masses (in russian)*, in Doklady Akademii Nauk, 1942, pp. 227–229.

[14] M. Kato, T. Teshima, and J. Honda, *Learning from positive and unlabeled data with a selection bias*, in Proceedings of the 7th International Conference on Learning Representations, New Orleans, LA, 2019.

[15] R. Kiryo, G. Niu, M. C. du Plessis, and M. Sugiyama, *Positive-unlabeled learning with non-negative risk estimator*, in Advances in Neural Information Processing Systems, Long Beach, CA, 2017, pp. 1675–1685.

[16] W. S. Lee and B. Liu, *Learning with positive and unlabeled examples using weighted logistic regression*, in Proceedings of the 20th International Conference on Machine Learning, Washington, DC, 2003, pp. 448–455.

[17] X. Li and B. Liu, *Learning to classify texts using positive and unlabeled data*, in Proceedings of the 18th International Joint Conference on Artificial Intelligence, Acapulco, Mexico, 2003, pp. 587–592.

[18] B. Liu, Y. Dai, X. Li, W. S. Lee, and P. S. Yu, *Building text classifiers using positive and unlabeled examples*, in Proceedings of the 3rd International Conference on Data Mining, Melbourne, FL, 2003, pp. 179–186.

[19] B. Liu, W. S. Lee, P. S. Yu, and X. Li, *Partially supervised classification of text documents*, in Proceedings of the 19th International Conference on Machine Learning, Sydney, Australia, 2002, pp. 387–394.

[20] C. McDiarmid, *On the method of bounded differences*, Surveys in Combinatorics, 141 (1989), pp. 148–188.

[21] M. Mehryar, R. Afshin, and T. Ameet, *Foundations of Machine Learning*, MIT Press, Cambridge, MA, 2012.

[22] F. Mordelet and J. P. Vert, *Prodige: Prioritization of disease genes with multitask machine learning from positive and unlabeled examples*, BMC bioinformatics, 12 (2011), pp. 1–15.

[23] V. Nair and G. E. Hinton, *Rectified linear units improve restricted boltzmann machines*, in Proceedings of the 27th International Conference on Machine Learning, Haifa, Israel, 2010, pp. 807–814.

[24] H. Ramaswamy, C. Scott, and A. Tewari, *Mixture proportion estimation via kernel embeddings of distributions*, in Proceedings of the 33rd International Conference on Machine Learning, New York City, NY, 2016, pp. 2052–2060.

[25] H. Shi, S. Pan, J. Yang, and C. Gong, *Positive and Unlabeled Learning via Loss Decomposition and Centroid Estimation*, in Proceedings of the 27th International Joint Conference on Artificial Intelligence, Stockholm, Sweden, 2018, pp. 2689–2695.

[26] V. Vapnik, *The nature of statistical learning theory*, Springer, New York, NY, 1999.

[27] H. Yu, J. Han, and K. C. Chang, *PEBL: positive example based learning for web page classification using SVM*, in Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Alberta, Canada, 2002, pp. 239–248.

[28] C. Zhang, Y. Hou, and Y. Zhang, *Learning from positive and unlabeled data without explicit estimation of class prior*, in Proceedings of the 34th AAAI Conference on Artificial Intelligence, New York, NY, 2020, pp. 6762–6769.

[29] T. Zhang and Z. Zhou, *Optimal margin distribution machine*, IEEE Transactions on Knowledge and Data Engineering, 32 (2019), pp. 1143–1156.

[30] Y. Zhang, L. Li, J. Zhou, X. Li, Y. Liu, Y. Zhang, and Z. Zhou, *POSTER: A PU learning based system for potential malicious URL detection*, in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Dallas, TX, 2017, pp. 2599–2601.